MMAPER

Accessible GIS Viewer

Comprehensive Technical Report

Generated: 07 August 2026
Platform: Windows (win32)
Project root: C:\Users\hp\Documents\MMAPER
Total source lines: 6697

Contents

- 1. Executive Summary
- 2. Project Overview & Purpose
- 3. Architecture
- 4. File Inventory
- 5. Data Model (data_model.py)
- 6. Layer Nesting Model (PDF-Bookmark Style)
- 7. Project Save / Load
- 8. Graphical User Interface (main.py)
- 9. Dialogs
- 10. Raster Creation & Editing
- 11. Spatial Analysis
- 12. Custom Layer & Feature Templates
- 13. Test Suite
- 14. How to Run
- 15. Dependencies
- 16. Code Statistics
- 17. Known Limitations & Suggested Next Steps
- Appendix: PROJ/GDAL Environment Fix

1. Executive Summary

MMAPER (Accessible GIS Viewer) is a desktop geographic information system built with Python and wxPython. It provides a keyboard-friendly, screen-reader-friendly interface for creating, importing, editing, analyzing and exporting both vector (GeoJSON / Shapefile / KML / GeoPackage) and raster (GeoTIFF / DEM) data. The application is structured as a tree of layers on top and a property/attribute table below.

Recent development replaced the older 'group' concept with a PDF-bookmark nesting model in which any layer can directly contain sub-layers. Layers carry their own editable properties (for example a planet's length or terrain). A position dialog asks, whenever a layer is already selected and a new layer is added, whether the new layer should be a child of the selected layer or a sibling. Whole projects (nesting + properties + features + raster pixels) can be saved to and loaded from a single JSON project file (.agis). Rasters can be created from scratch and edited with operations such as fill, rectangle fill, add/multiply, invert, gradient, noise and smoothing.

The project ships with an automated test suite of seven scripts covering the vector workflow, the nesting model, project save/load, raster editing, the spatial analysis engine, the template engine and the field-list editor. All tests pass. The implementation totals 6697 lines of Python across the two core modules.

The latest additions are an on-screen map canvas and a legend control. A matplotlib-based live map pane sits inside a MapCanvasPanel and renders every visible layer with its own symbolization (colour, line width, fill opacity, marker type and marker size). A list-based legend beside the canvas shows every layer in draw order with a checkbox per layer for show/hide, double-click/button toggles, and Move Up / Move Down buttons (also available as Alt+Up / Alt+Down and from the tree context menu) to control the layer draw order (z-order). The pane can be shown or hidden from the View menu (Ctrl+M), and each vector layer's style is edited through a Layer Style dialog and saved with the project.

Vector styling is now attribute-driven. The Layer Style dialog supports three modes: a single symbol style, graduated choropleth rendering by a numeric field over a user-selectable colour map, and categorised rendering that maps each distinct text value to its own colour. Terrain rasters are drawn through the same colour-map machinery, with a dedicated Raster Style dialog (Raster menu, Ctrl+Alt+M) choosing the colour map used on the canvas.

Large rasters are handled without degrading the interface. When a raster exceeds 32 million cells it is decimated into a reduced working copy at load time (the original dimensions are preserved in the layer summary), and canvas rendering always downsamples to a 1024-pixel maximum so panning and zooming stay responsive. Raster bands embedded in project files are now compressed (zlib + base64), which keeps .agis files small for big rasters while remaining fully backward-compatible with version 1 projects.

Rasters now support multiple bands. Every TerrainLayer carries a list of independent bands with their own names; editing operations always target the active band, and a Manage Bands dialog (Raster menu, Ctrl+Alt+B) lets the user set the active band, add, duplicate, rename and remove bands. Band names and the active band are persisted in project files and round-trip through GeoTIFF save/load. Raster previews, the raster style dialog and the live map canvas can render any three bands as an RGB composite (per-band 2%-98% percentile stretch), or a single band through a colour map.

Spatial analysis now goes beyond single-feature edits. A SpatialAnalysisEngine in data_model.py performs multi-criteria Resource Deposit Suitability Analysis (distance-to-infrastructure proximity combined with optional terrain slope/elevation exclusions), Organisational Network Influence & Reach analysis (multi-ring proximity buffers, Voronoi/Thiessen cells, and density heat grids), and Corridor Vulnerability assessment (flagging transport corridors that pass within a risk radius of an organisational node). Network Cost Surfaces extends the engine from straight-line great-circle proximity to real cost-weighted pathfinding: any raster band can be used as a cost surface and least-cost paths (Dijkstra over the cell grid, with NaN cells as barriers) connect two points, radiate a cumulative-cost surface, or wire a whole set of points into a chain, star or minimum-cost spanning

network. Resource Deposit Suitability can now blend its great-circle proximity score with that same cost-weighted network reach: a user-supplied cost surface and a 0-1 weight turn the overlay into a cost-aware siting model, with network_reach_cost / network_reach_score fields attached alongside the proximity fields. Three dialogs - Resource Suitability Analysis, Org Network Reach and Network Cost Surfaces - sit on the Spatial menu. Every run attaches calculated fields (suitability score, nearest-org distance, control zone, corridor risk) directly to the source features and refreshes the live map canvas, so the results are immediately visible in the tree, the attribute list and the legend.

Layers and features can now be reused through a standardized template engine. The built-in field presets were expanded from 6 to 23, and any layer or feature can be saved as a reusable custom template that lives in mmapper_templates.json and appears in the 'Field Template' and 'Add Fields from Template' dialogs. Templates can be exported to, and imported from, JSON files. All templates share one canonical schema (a 'mmapper_template' version marker, a layer/feature kind, a name, a description, and a normalized list of fields with canonical types text/number/integer/boolean), so a template written on one machine opens identically on another.

2. Project Overview & Purpose

The application targets accessibility first: the main window uses a vertical splitter with a wx.TreeCtrl (layer navigator) on top and a wx.ListCtrl (property/attribute reader) below. The user rejected spreadsheet grids in favour of list-based, single-selection controls that work well with screen readers and keyboard navigation.

Core capabilities:

- Vector editing: create layers, add/edit/duplicate/delete features (Point, LineString, Polygon), edit geometry, manage a field schema, add/remove/rename fields, apply field templates.
- Attribute workflows: add/edit feature properties, split coordinates into X/Y fields, compute area/length and centroid fields, copy features between layers.
- Spatial analysis: buffer and simplify individual features, reproject layers between CRS (pyproj), set layer CRS.
- Multi-criteria spatial analysis: resource deposit suitability scoring (distance to infrastructure plus optional terrain slope/elevation exclusions and optional cost-weighted network reach over a raster cost surface), organisational influence zones (multi-ring buffers, Voronoi/Thiessen cells, density heat grids), and corridor vulnerability flags (routes within a risk radius of an org node); all results are written back to feature attributes and rendered on the map.
- Layer nesting: PDF-bookmark style parent/child relationships between any layers, with a radio-box position prompt when adding layers.
- Layer visibility: hide/show any layer (Layer menu, tree context menu, or shortcut V); hidden layers are greyed out in the tree, excluded from the HTML web-map export, and restored on project reload.
- Live map canvas: a matplotlib map pane renders every effectively-visible layer on screen with per-layer symbolization, auto-scales to the data bounds, handles geographic (latitude/longitude) aspect, and can be toggled on/off from the View menu (Ctrl+M).
- Map legend control: a list beside the canvas lists all layers in draw order with a checkbox per layer, a 'Visible' column, double-click/button visibility toggles and Move Up / Move Down buttons; the legend stays in sync with the tree selection.
- Layer draw order (z-order): Move Layer Up / Move Layer Down (Layer menu Alt+Up/Alt+Down, tree context menu, or the legend buttons) reorder a layer among its siblings; the change is undoable and the canvas re-renders top-down.

- Layer styling: a Layer Style dialog sets a vector layer's colour, line width, fill opacity, marker type and marker size, and supports attribute-driven graduated (numeric field over a colour map) and categorised (unique values) modes; styles are persisted in the project and displayed in the layer summary.
- Layer properties: every layer can carry its own name/value/type properties (e.g. planet length, terrain, phase).
- Templates: 23 built-in field presets plus user-defined custom templates; any layer or feature can be saved as a reusable custom template (stored in mmapper_templates.json) or exported to a standardized JSON file, and standardized (or legacy field-dict) JSON templates can be imported to create a new layer or add fields to a selected layer/feature.
- Right-click context menu on the layer tree: rename, duplicate, remove, properties, fields, save/export and feature operations.
- Drag-and-drop re-nesting: drag any layer onto another layer to make it a child (or onto the root to make it top-level), with cycle protection and undo support.
- Undo/redo: snapshot-based undo/redo (Ctrl+Z / Ctrl+Shift+Z) covering vector edits, raster operations, layer/property changes and nesting.
- Raster workflow: open existing GeoTIFF/DEM rasters, create new ones from scratch, edit them with operations, sample elevation at a point, save back to GeoTIFF.
- Project persistence: save/load the entire workspace (layers, nesting, layer properties, embedded vector features and raster pixel arrays) as one .agis JSON file.
- Export: interactive Folium HTML web map and CSV attribute table export.

3. Architecture

The application is intentionally kept to two Python modules plus a set of standalone test scripts. There is no framework; the GUI is plain wxPython and the data layer is built directly on geopandas / shapely / rasterio / numpy.

Module	Responsibility	Lines
data_model.py	Data classes (Layer, Feature, MapLayer, TerrainLayer), raster editing operations, project (de)serializa	2036
main.py	wxPython GUI: window, tree/list, menus, dialogs, all event handlers	4461

Import chain: main.py imports the data classes and the save_project / load_project helpers from data_model.py. data_model.py sets the PROJ_LIB and GDAL_DATA environment variables before importing rasterio/geopandas so that the bundled GDAL/PROJ data can be located on Windows. geopandas, shapely, pyproj and rasterio provide the spatial engine; pandas/numpy provide tabular and array computation; matplotlib, folium and contextily support export/rendering; scipy provides the convolution kernel used for raster smoothing and the Voronoi tessellation used by the organisational reach analysis.

Data flow for a typical action (e.g. Add Feature):

- User picks a menu item -> the bound wx.EVT_MENU handler runs.
- The handler shows a modal dialog collecting input.
- The handler calls a method on the selected data object (MapLayer / TerrainLayer / Feature).
- The handler refreshes the tree node and repopulates the property ListCtrl.

- The status bar confirms the action.

4. File Inventory

File	Lines	Description
data_model.py	2236	Data classes and persistence.
main.py	4461	GUI, menus, dialogs, handlers.
_smoke_test.py	362	Vector/attribute/spatial test.
_nesting_test.py	191	Layer nesting + properties test.
_project_test.py	168	Project save/load test.
_raster_test.py	350	Raster create/edit test.
_spatial_analysis_test.py	438	Spatial analysis engine + dialogs test.
_template_test.py	290	Template engine + template dialogs test.
_fields_test.py	155	Field list value editing + unit parsing test.
requirements.txt	22	Python dependency list.
Launch Mapper.bat	4	Windows launcher using venv python.
sample.geojson	39	Sample vector data.
sample_dem.tif	1	Sample terrain raster.
_report.py	-	This report generator.

5. Data Model (data_model.py)

The data layer is organised around a small class hierarchy. All layers derive from the Layer base class, which provides the PDF-bookmark nesting structure and layer-level properties. There are 5 classes and 31 module-level helper functions.

Layer

Methods: __init__, style_with_defaults, set_style, add_property, remove_property, rename_property, contains, all_members, is_effectively_visible, to_dict

Feature

Methods: __init__, get_name, get_geometry_summary, to_geojson

MapLayer

Methods: __init__, new_empty, from_geojson, from_vector, infer_schema, _guess_type, set_crs, add_field, remove_field, rename_field, add_feature, remove_feature, duplicate_feature, save_to_geojson, to_dict, from_dict, to_dataframe, to_geodataframe, reproject, buffer_feature, simplify_feature, add_geometry_fields, add_centroid_field, coordinate_properties, split_geometry_coordinates, split_property_coordinates, get_attribute_summary, compute_area_sq_units, get_geographic_summary

TerrainLayer

Methods: __init__, _load, set_active_band, _ensure_band_names, add_band, duplicate_band, remove_band, rename_band, band_stats, rgb_array, recompute_stats, new_empty, _set_geometry, save_to_geotiff, fill, fill_rect, add_constant, multiply_constant, invert, add_gradient, add_noise, smooth, sample_elevation, get_geometry_summary, to_dict, from_dict

SpatialAnalysisEngine

Methods: _sample_at, _terrain_slope_pct, _attach_fields, _classify_zone, compute_resource_suitability, generate_org_influence_zones, calculate_corridor_vulnerability, _cost_array, _snap_to_grid, _corridor_source_cells, _dijkstra_from, least_cost_path, cost_accumulation, least_cost_network

Class responsibilities

Class	Purpose	Key attributes
Layer	Base for all layers; provides nesting (children/parent), layer properties (pr	name, children, parent, properties, style
Feature	One vector feature: type, geometry (GeoJSON dict), properties.	type, geometry, properties
MapLayer	A vector layer: features, schema, CRS, source file; import/export and geo	features, crs, filepath, schema
TerrainLayer	A raster/terrain layer holding one or more in-memory float64 pixel bands;	bands, res, active_band, selection, sta
SpatialAnalysisEngine	Vector + raster multi-criteria analysis: resource suitability scoring (great-circ	compute_resource_suitability, gener

Notable methods on Layer (base class)

Method	Purpose
style_with_defaults()	Return the layer's style merged over DEFAULT_STYLE (colour, line_width, fill_alpha, mar
set_style(style)	Replace the layer's style with the given keys (unspecified keys reset to defaults).
contains(other) / all_members()	Cycle-safe nesting helpers: descendant check and flattened member list.
add_property / remove_property / rename_property	Manage the layer's own name/value/type properties.

Notable methods on MapLayer

Method	Purpose
new_empty(name, crs, schema)	Create a blank vector layer.
from_geojson / from_vector	Import a layer from GeoJSON or any vector format readable by geopandas.
add_field / remove_field / rename_field	Manage the layer schema and propagate to features.
add_feature / remove_feature / duplicate_feature	Feature lifecycle operations.
save_to_geojson	Write the layer to a GeoJSON FeatureCollection.
reproject(target_crs)	Reproject all geometries (pyproj).
buffer_feature / simplify_feature	Single-feature spatial operations (shapely).
add_geometry_fields / add_centroid_field	Compute area/length and centroid attributes.
split_geometry_coordinates / split_property_coordinates	Extract X/Y columns.
get_attribute_summary / get_geographic_summary	Numeric summaries (totals, means, density).
to_dataframe / to_geodataframe	Convert to pandas / geopandas structures.

TerrainLayer raster operations

Method	Purpose
new_empty(name, crs, width, height, bounds, resolution, fill)	Create a raster batch with a starting value and the given number of bands.
save_to_geotiff(filepath)	Write all in-memory bands to a GeoTIFF.
fill(value)	Set every cell of the active band to a value.
fill_rect(r0, r1, c0, c1, value)	Set a rectangular block of cells (row/col) in the active band.
add_constant / multiply_constant	Arithmetic on all cells of the active band (e.g. raise/lower terrain).
invert()	Flip elevation in the active band: new = min + max - old.
add_gradient(direction, start, end)	Add a linear slope to the active band; 6 directions (N/S/E/W/diagonals).
add_noise(amount)	Add Gaussian noise (standard deviation) to the active band.
smooth(passes)	3x3 averaging kernel (scipy convolution) on the active band, repeated.
set_active_band(index)	Make another band the editing target; returns False if out of range.
add_band(value, name) / duplicate_band(index)	Append a new empty band or a copy of an existing band.

Method	Purpose
remove_band(index)	Delete a band (at least one band is kept) and keep active-band/data in sync.
rename_band(index, name)	Give a band a readable name (defaults to 'Band N').
band_stats(index)	Min/max/mean/std/range for any single band.
rgb_array(r, g, b)	Stack three bands into a float32 RGB array with a per-band 2%-98% percentile stretch
sample_elevation(lon, lat)	Read a cell value from the active band via the affine transform.
recompute_stats()	Recalculate min/max/mean/std/range for the active band after any edit.

Template engine (module-level functions)

The template engine standardizes layer and feature templates so they can be shared between devices. Every template is validated and normalized into one canonical JSON shape carrying a 'mmapper_template' version marker, a kind ('layer' or 'feature'), a name, a description, and a 'fields' list whose types are limited to text, number, integer or boolean (defaults are coerced to the field type). Layer templates also capture the layer CRS, its own properties and its style. For convenience a legacy {field: type} dict is accepted on import and wrapped into a layer template.

Function	Purpose
build_layer_template(layer, name, description)	Build a standardized layer template from schema + properties + style.
build_feature_template(layer, feature, name, description)	Build a feature template from a feature's properties and the layer schema.
validate_template(data)	Normalize a template dict (or legacy field dict) into the canonical format; raise
template_to_json_string / template_to_json	Serialize a template to a JSON string / file (indented, deterministic).
template_from_json_string / template_from_json	Parse and validate a template from a JSON string / file.
create_layer_from_template(template, name, crs)	Create a new MapLayer from a layer template (schema, properties, style).
apply_feature_template(layer, feature, template)	Add a feature template's fields to a layer schema (and feature defaults); retur
load_custom_templates / save_custom_templates	Read/write the on-disk custom template library (mmapper_templates.json).

6. Layer Nesting Model (PDF-Bookmark Style)

The previous implementation used a separate LayerGroup object. It was replaced by a direct nesting model where any Layer can be a parent or a child - exactly like the bookmarks of a PDF document. There is no separate 'group' type.

How it works:

- Every Layer has a list of children and a pointer to its parent (parent is None for top-level layers).
- Layer.contains(other) reports whether 'other' is this layer or any of its descendants (used to prevent cycles).
- Layer.all_members() flattens all descendants.
- The wx.TreeCtrl is rebuilt recursively from top-level layers via _create_tree_item.
- Frame.all_layers() returns every layer (top-level and nested) for operations such as feature lookup.

Placement dialog:

- Adding the first layer: nothing is selected, so no prompt appears and the layer becomes top-level.
- Adding a layer while a layer is selected: a PositionDialog radio box asks whether the new layer is 'Child of the selected layer' or 'Top level (sibling of the selected layer)'.
- Sibling placement goes to the same parent as the selected layer (top level if the selected layer is top level).
- Removing a layer detaches it from its parent (frame._detach) and warns if it still has sub-layers.
- Nested layers appear indented under their parent in the tree, and the parent summary shows a 'Sub-Layers' count.

7. Project Save / Load

A project is a single JSON file with extension .agis. It stores the complete workspace so it can be reopened exactly as it was, without depending on the original source files.

Feature	How it is stored
Layer hierarchy	Recursive 'children' arrays in every layer's to_dict; parent pointers are restored on load.
Layer properties	'properties' and 'prop_types' dictionaries on each layer.
Layer visibility	Every layer's 'visible' flag is stored, so hidden sub-trees stay hidden on reload.
Layer style	Every layer's 'style' dict is stored, so symbolization (colour, line width, fill, marker, colour map, attribute
Vector features	Embedded GeoJSON FeatureCollection per MapLayer (self-contained).
Raster layers	Each band is embedded in compressed form (zlib + base64 of the raw float64 bytes), plus filepath, CR
Project version	Files carry 'project_version': 2; version 1 projects (uncompressed list-of-lists bands) are still read trans
Terrain with existing file	On load, re-reads the GeoTIFF when the file still exists; otherwise uses embedded data/metadata.
Validation	load_project rejects files whose type is not 'AccessibleGISProject'.

API: save_project(filepath, layers) writes the JSON; load_project(filepath) returns the reconstructed top-level layer list. Each layer serializes itself via to_dict() and is rebuilt by _layer_from_dict() which dispatches on the 'kind' field (MapLayer, TerrainLayer, or base Layer).

Menu: File -> New Project (Ctrl+Alt+N), Open Project (Ctrl+Alt+O), Save Project (Ctrl+Alt+S), Save Project As (Ctrl+Alt+Shift+S). Opening or starting a new project prompts for confirmation when the current project is non-empty.

8. Graphical User Interface (main.py)

main.py contains 25 wxPython classes and 237 methods across dialogs and the main frame.

MainFrame layout

MainFrame is a wx.Frame titled 'Accessible GIS Viewer' (840x640). A horizontal BoxSizer holds a MapCanvasPanel on the left and a vertical SplitterWindow on the right. The MapCanvasPanel itself is split: a legend panel (a wx.ListCtrl with checkboxes, 'Visible' column and Move Up / Toggle Visibility / Move Down buttons) on the left, and the matplotlib FigureCanvasWxAgg with a navigation toolbar for zoom/pan on the right. The SplitterWindow has two panels: a wx.TreeCtrl (hidden root, expand buttons) used as the layer navigator, and a two-column wx.ListCtrl (Property | Value) showing either a feature's attributes, a layer summary, or a terrain summary. Selecting a tree node repopulates the list and highlights the same layer in the legend; double-clicking a property row opens an editor dialog. The legend and canvas are refreshed automatically after any change to layers, features, visibility, style or draw order, and can be hidden entirely via the View menu.

File menu

Item	Shortcut	Action
New Project	Ctrl+Alt+N	Clear the workspace.
Open Project...	Ctrl+Alt+O	Load a .agis project.
Save Project	Ctrl+Alt+S	Save the whole project.
Save Project As...	Ctrl+Alt+Shift+S	Save project to a new file.
New Layer...	Ctrl+Shift+N	Create an empty vector layer.
Open GeoJSON...	Ctrl+O	Import a GeoJSON file.
Add Vector Layer...	Ctrl+Shift+O	Import SHP/KML/GPKG/etc.
Open Terrain Raster...	Ctrl+T	Open a GeoTIFF/DEM raster.
Save	Ctrl+S	Save current vector layer or raster.
Save As...	Ctrl+Shift+S	Save current layer/raster to a new file.
Export Web Map (HTML)...	Ctrl+E	Folium interactive map export.
Export Layer Table (CSV)...	Ctrl+D	Attribute table to CSV.
Exit	Ctrl+Q	Close the application.

Edit menu

Item	Shortcut	Action
Add Feature...	Ctrl+N	Add point/line/polygon feature.
Edit Geometry...	Ctrl+G	Edit geometry via coordinates.
Duplicate Feature	Ctrl+Shift+D	Clone selected feature.

Item	Shortcut	Action
Delete Feature	Delete	Remove selected feature.
Add/Edit Property...	Ctrl+P	Add or change a feature property.
Copy Feature to Layer...	Ctrl+Shift+C	Copy feature into another layer.
Query Elevation...	Ctrl+L	Sample the selected terrain layer.

View menu

Item	Shortcut	Action
Map Pane (checkable)	Ctrl+M	Show or hide the live on-screen map canvas and its legend; hidden p

The legend is part of the MapCanvasPanel, so it appears whenever the Map Pane is shown. It lists every layer (top-level and nested, in all_layers() order) with a checkbox and a 'Visible' column, colours the layer name with the layer's style colour (grey when effectively hidden), and mirrors the tree selection in both directions.

Layer menu

Item	Shortcut	Action
Layer Properties...	-	Edit properties on the layer itself.
Show/Hide Layer	V	Toggle whether the layer is visible (greyed out in the tree, skipped in
Layer Style...	Ctrl+Shift+Y	Customise colour, line width, fill opacity, marker type/size, or switch t
Move Layer Up	Alt+Up	Raise the selected layer in the draw order (re-renders the map; undo
Move Layer Down	Alt+Down	Lower the selected layer in the draw order (re-renders the map; undo
Remove Layer	-	Remove selected layer (warns on sub-layers).
Rename Layer...	F2	Rename the selected layer.
Duplicate Layer	-	Deep-copy the layer.
Set CRS...	-	Assign a coordinate reference system.
Reproject Layer...	-	Transform to a new CRS.
Add Field...	-	Add a schema field.
Remove Field...	-	Remove a schema field.
Add Fields from Template...	-	Add preset fields (Demographics, Economy, etc.) or saved custom te
Fields List (View/Edit)...	Ctrl+F	List and edit all field values.
Export Layer as Template (JSON)...	-	Write the selected layer (schema, CRS, properties, style) as a standa
Export Feature as Template (JSON)...	-	Write the selected feature's fields and defaults as a standardized JSO
Import Template (JSON)...	-	Import a standardized JSON template: creates a new layer, or adds a
Save Layer/Feature as Custom Template...	-	Store the current layer or feature in the reusable custom template libr
Manage Custom Templates...	-	List, export to JSON or delete the saved custom templates.

Item	Shortcut	Action
Add Area/Length Fields	-	Compute area/length per feature.
Add Centroid Field	-	Compute centroid per feature.

Raster menu

Item	Shortcut	Action
New Terrain/Raster...	Ctrl+Shift+T	Create a raster from scratch (dialog; number of bands is configurable
Edit Raster Values...	Ctrl+R	Operations-based editing of the selected raster.
Preview Raster (Image)...	Ctrl+Shift+P	Matplotlib preview with band selection and single/RGB rendering.
Raster Style...	Ctrl+Alt+M	Choose the colour map or an RGB composite (three band pickers) us
Manage Bands...	Ctrl+Alt+B	Add, duplicate, rename, remove or set the active band for the selecte

Spatial menu

Item	Action
Buffer Selected Feature...	Buffer the selected feature by a distance.
Simplify Selected Feature...	Simplify the selected feature geometry.
Split Selected Feature Coordinates...	Add X/Y columns for the selected feature.
Split All Features Coordinates	Add X/Y columns for every feature.
Resource Suitability Analysis...	Score each deposit point by proximity to transport corridors, optionally excluding steep / out-o
Org Network Reach...	Compute organisational influence via multi-ring buffers, Voronoi cells or a density heat grid; a
Network Cost Surfaces...	Connect points by least-cost paths over a raster cost surface: pair paths, ordered chains, star

9. Dialogs

main.py defines 23 dialog classes (plus MainFrame and MapCanvasPanel). All dialogs are keyboard-operable wx.Dialog subclasses using standard OK/Cancel or explicit Apply/Close buttons.

Dialog	Used for	Key controls
DetailDialog	View/edit a single property value.	Multiline text, OK/Cancel.
PropertyDialog	Add or edit a feature property (name, type, value).	Text/choice fields.
SingleFieldDialog	Add one schema field to a layer (text, number, integer or boolean).	Name + type choice.
NewLayerDialog	Create a vector layer; pick template and build schema.	ListCtrl of fields + Add/Remove.
NewRasterDialog	Create a raster: name, CRS, width/height, bounds, start value, number of bands.	Form fields.
RasterEditDialog	Operations-based raster editing.	Operation radio box, band choice, dynamic params, stats list.
RasterPreviewDialog	Preview a raster as an image before saving.	Render-mode choice (single band / RGB composite), band pickers, matplotlib image canvas.
RasterStyleDialog	Choose how a terrain raster is rendered on the map canvas.	Colour map choice, render-mode choice (single band / RGB composite) with R/G/B band pickers, live preview canvas.
BandAddDialog	Enter how many bands to add to a raster.	Number text field, current count display.
BandManagerDialog	Manage a raster's bands: set active, add, duplicate, rename, remove.	ListCtrl of bands (Band | Name | Min/Max/Mean | Active) plus Set Active / Add Band(s) / Duplicate / Rename / Remove buttons.
AddFeatureDialog	Add a Point/LineString/Polygon feature by coordinates.	Geometry type radio, coordinate text area.
GeometryEditDialog	Edit an existing feature geometry.	Points list + paste parser.
LayerChoiceDialog	Choose the target layer (e.g. Copy Feature).	Layer list.
SplitCoordinatesDialog	Choose X/Y field names for splitting.	Form fields.
TemplateDialog	Add fields from a template (built-in presets or saved custom templates).	Template choice + live field preview.
TemplateNameDialog	Enter a name and description when exporting or saving a template.	Name + description text fields.
TemplateLibraryDialog	Manage the saved custom template library (export to JSON, delete).	ListCtrl of templates + Export/Delete buttons.
PositionDialog	PDF-bookmark placement when adding a layer.	Radio box (sibling vs child).
FieldListDialog	List/edit all fields of a feature/layer, or layer properties.	Feature picker combo, ListCtrl with inline editing.
LayerStyleDialog	Set a vector layer's symbolization for the map canvas.	Colour picker, line width, fill opacity, marker choice, size, live preview, plus graduated/categorised attribute modes with field, colour map and class-count controls.

Dialog	Used for	Key controls
SuitabilityAnalysisDialog	Resource deposit suitability scoring: pick the deposit points layer, an optional transport corridors layer, an optional terrain layer, an optional cost surface (raster band) and a 0-1 network reach weight, a max distance and an optional max slope.	Deposit/corridor/terrain layer choices, optional cost-surface + band choices, network reach weight (0-1), max distance (km) and max slope (%) text fields.
OrgReachDialog	Organisational network influence & reach: pick the org points layer, an analysis method (multi-ring buffers, Voronoi cells or density heat grid), buffer rings, an optional corridors layer and a risk radius.	Org layer + method choices, buffer rings text, corridors choice, risk radius (km).
NetworkCostDialog	Network cost surfaces: connect points by least-cost paths over a raster cost surface (pair / chain / star / tree modes).	Points + cost-surface layer choices, cost-band choice, connect-method choice, from/to point pickers (pair mode), diagonal-moves checkbox.

FieldListDialog has three modes: a feature's attributes, a layer's fields, and a layer's own properties (opened from Layer -> Layer Properties). In property mode it uses add_property/remove_property/rename_property on the Layer and coerces values according to the declared type (number, integer, boolean, text). In feature and layer modes a Feature: combo box lists every feature in the layer, so a feature's values can be edited without pre-selecting it in the main tree; when the dialog is opened for a layer with no feature selected the first feature is pre-selected so values are immediately editable. Numeric values accept unit suffixes (e.g. '120 m2', '1,250 km2', '7 units') - the leading number is stored and any unit text is ignored - so fields like area stay numeric.

10. Raster Creation & Editing

TerrainLayer now owns an in-memory float64 pixel array (data), an affine transform, bounds, resolution and statistics. This lets the user create rasters from scratch and edit them without touching the source file until Save.

Creation (Raster menu -> New Terrain/Raster):

- A NewRasterDialog collects a name, CRS (default EPSG:4326), width and height in cells (defaults 100x100), an optional geographic bounds string 'left, bottom, right, top', a starting value (default 0 = flat terrain) and a number of bands (default 1).
- Bounds left blank derive from the dimensions at resolution 1x1.
- The new raster appears in the tree through the same PDF-bookmark position prompt as other layers.
- It is marked 'Not saved yet' until File -> Save As writes it to a GeoTIFF.

Editing (Raster menu -> Edit Raster Values):

- A RasterEditDialog shows an Operation radio box: Fill All, Set Rectangle (Rows), Add Constant, Multiply by, Invert, Add Gradient, Add Noise, Smooth.
- Only the parameters needed for the selected operation are shown (rows hide/show).
- Apply runs the operation immediately on the active band, updates the layer statistics, refreshes the main window and keeps the dialog open for chained operations.
- A statistics ListCtrl inside the dialog shows live min/max/mean/std/range, raster size and file state.
- Because sample_elevation now reads the in-memory array, elevation queries reflect edits even before the file is saved.

Bands (Raster menu -> Manage Bands):

- Every TerrainLayer holds a list of independent bands (float64 arrays) with names; the active band is the one edited and shown in the summary.
- The Manage Bands dialog (also on the layer's right-click menu) lists every band with its name, min/max/mean statistics and active marker, and offers Set Active, Add Band(s), Duplicate Band, Rename Band and Remove Band.
- Operations are undoable as a group; band names and the active band are saved in the project and survive GeoTIFF save/load.

Rendering:

- The map canvas draws terrain rasters through the same colour-map pipeline used for vector layers, so a DEM is shown as a shaded surface rather than a flat grey block.
- A Raster Style dialog (Raster menu -> Raster Style, Ctrl+Alt+M) picks the colour map, or switches to an RGB composite by choosing any three bands as the red/green/blue channels; the choice (render mode, colour map, RGB band triplet) persists in the layer's style dict and in the project.
- The Preview dialog offers the same single-band / RGB choice so bands can be inspected before saving.

11. Spatial Analysis

The SpatialAnalysisEngine in data_model.py adds multi-criteria spatial analysis on top of the existing single-feature tools. It works on lon/lat (EPSG:4326) coordinates, uses great-circle distances (haversine) for proximity, approximates buffer circles geodetically, and clips Voronoi cells to the data bounding box. Every analysis attaches calculated fields to the source features (so the results appear in the field schema, the attribute list and the CSV export) and the handlers refresh the tree and the live map canvas automatically.

Resource Deposit Suitability Analysis

compute_resource_suitability(deposits, transport_layers, terrain_layer, max_distance_km, max_slope_pct, min_elevation, max_elevation, cost_layer, cost_band, cost_weight, diagonal, max_reach_cost) scores each deposit point from 0 to 1 by its distance to the nearest transport corridor (1.0 = on the corridor, 0.0 = at or beyond max_distance_km). When a terrain raster is supplied it also samples elevation and a slope raster computed from the DEM; deposits on ground steeper than max_slope_pct or outside the elevation window are masked out (suitability_excluded = True, score 0). When a cost surface raster and a network reach weight are supplied the score becomes a weighted blend of the proximity score and the cost-weighted network reach: one multi-source Dijkstra run radiates from every transport corridor cell (the corridors are rasterized at half-cell spacing) and each deposit reads the accumulated traversal cost at its own cell. Reach costs are normalized to 0..1 using max_reach_cost when given, otherwise the largest finite reach cost observed in the run; cost_weight 0 means pure proximity, 1.0 means pure network reach, and in between the two are blended. Attached fields: suitability_score, distance_to_infra_km, suitability_excluded, network_reach_cost / network_reach_score when cost scoring is active, and elevation_m / slope_pct when terrain is used. The method returns a summary dict with analyzed / excluded counts and best / min / max / mean scores.

Organisational Network Influence & Reach

generate_org_influence_zones(org, method, buffer_rings_km, density_cells) always attaches nearest_org_km (great-circle distance to the closest other org) and org_control_zone (classified into rings such as 'Zone 3 (< 100 km)'). Three methods are supported: 'buffers' creates a polygon layer of multi-ring proximity buffers around each org point; 'voronoi' builds a Thiessen-cell polygon layer (with area_km2 per cell) using

scipy.spatial.Voronoi; 'density' produces a density heat grid (numpy histogram2d) plus its geographic extent. The polygons layer is added to the tree and map automatically.

Corridor Vulnerability

calculate_corridor_vulnerability(corridor, org, risk_radius_km) measures each transport corridor's closest approach to any organisational node. Corridors passing within the risk radius are flagged (corridor_vulnerable = True) and the nearest org name and distance are attached. The summary reports how many of the total corridors are vulnerable and their names. In the Org Network Reach dialog it runs in the same pass, so a single operation reports both the influence zones and the at-risk routes.

Network Cost Surfaces

Where the methods above measure straight-line great-circle proximity, Network Cost Surfaces instead weights every raster cell with a traversal cost and finds paths that accumulate the least cost across the grid (Dijkstra's algorithm over the cell graph; moving between cells costs the average of the two cells' values, scaled by sqrt(2) for diagonal moves, and non-finite cells - GeoTIFF nodata - are impassable barriers). least_cost_path(cost_layer, start, end) snaps the two points to the grid, finds the cheapest route and returns a LineString plus cumulative cost, geodesic length and cell count. cost_accumulation(cost_layer, source) radiates a cumulative-cost (cost-distance) surface from a source point - the raster analogue of the proximity buffers. least_cost_network(cost_layer, points, mode) connects a set of points with least-cost paths in three modes: 'chain' joins them in order, 'star' joins every point to the first, and 'tree' builds a minimum-cost spanning network (a Kruskal minimum spanning tree over the all-pairs least-cost graph). The Network Cost Surfaces dialog (Spatial menu) picks the points layer, cost raster and band, chooses the connect method, and for a pair path selects the from/to features; the resulting line layer is added to the tree and map with from_node / to_node / path_cost / path_distance_km / cells fields. Pathfinding is capped at 4 million cells so it stays responsive on the app's reduced working rasters.

12. Custom Layer & Feature Templates

Templates let the user reuse a layer's schema (or a feature's properties) without re-entering fields by hand. The 23 built-in presets in FIELD_TEMPLATES cover domains such as Demographics, Economy, Terrain & Nature, Infrastructure, Administrative, Health Services, Education, Transport Network, Utilities & Energy, Water Resources, Emergency Services, Agriculture & Forestry, Environment & Conservation, Mining & Resources, Urban Development, Climate & Weather, Security & Defence, Trade & Commerce, Telecommunications, Housing & Real Estate, Heritage & Culture and Tourism & Recreation. On top of these, any layer or feature can be saved as a custom template.

Where templates are used

- New Layer dialog: the 'Field Template' dropdown lists every built-in preset plus every saved custom template; 'Apply Template' fills the schema list.
- Layer -> Add Fields from Template: adds the selected template's fields (built-in or custom) to the current layer.
- Layer -> Manage Custom Templates: lists the saved library, and can export a template to a JSON file or delete it.

Exporting

- Layer -> Export Layer as Template (JSON)... writes the selected layer's schema, CRS, layer properties and style as a standardized JSON template.

- Layer -> Export Feature as Template (JSON)... (also on the feature's right-click menu) writes the selected feature's property fields and their default values.
- Every export prompts for a template name and description, and validates the output before writing.

Importing

- Layer -> Import Template (JSON)... reads a standardized template file. A layer template is instantiated as a new layer (inserted at the usual position prompt); a feature template adds its fields to the selected layer's schema and applies defaults to the selected feature.
- Legacy JSON written as a plain {field: type} object is accepted and wrapped into a layer template automatically.
- All field types are normalized to text, number, integer or boolean, so templates are interchangeable between machines.

Custom library

- Layer -> Save Layer/Feature as Custom Template... stores the current layer (or selected feature) in mmapper_templates.json, which is a standardized JSON file that can also be edited by hand or copied between installations.
- Saved templates appear immediately in the New Layer and Add Fields from Template dialogs; the library survives application restarts.

13. Test Suite

Seven standalone scripts exercise the application end to end. Each creates a real MainFrame (so tree/list behaviour is tested), prints progress lines, and exits non-zero on any assertion failure. All currently pass.

Script	Covers	Status
_smoke_test.py	Feature add (Point/Line/Polygon), paste parser, field templates, schema rename/add/remove, coordina	PASS
_nesting_test.py	Seven sub-layers under a parent, contains()/detach, parent pointers, layer summary with properties, pr	PASS
_project_test.py	save_project/load_project roundtrip, nested children + features, TerrainLayer from file and missing-file	PASS
_raster_test.py	new_empty creation, fill, fill_rect, add/multiply, gradient, invert, noise+smooth, elevation sampling in m	PASS
_spatial_analysis_test.py	module-level compute_spatial_suitability scoring, SpatialAnalysisEngine.compute_resource_suitability	PASS
_template_test.py	23-preset field template library with canonical types, build_layer_template / build_feature_template out	PASS
_fields_test.py	_parse_number tolerant numeric parsing (unit suffixes such as m2/km2/square units and thousands se	PASS

How to run a test (from the project folder): venv\Scripts\python.exe _raster_test.py

The tests are independent scripts (not a pytest suite); they intentionally avoid depending on pytest so they can be run with the bare venv interpreter.

14. How to Run

- Double-click 'Launch Mapper.bat' (preferred). It cd's to the project folder and runs venv\Scripts\python.exe main.py, pausing on errors.
- Manually: venv\Scripts\python.exe main.py from the project directory.
- The system-wide Python is NOT used because it lacks the GIS libraries; everything runs inside the local venv.
- Set up a fresh environment with: python -m venv venv and venv\Scripts\pip install -r requirements.txt

15. Dependencies

Package	Role
wxPython>=4.2.1	Desktop GUI toolkit.
geopandas>=0.14.0	Vector IO and CRS operations.
shapely>=2.0.0	Geometry buffer/simplify/area/centroid.
pyproj>=3.6.0	CRS parsing and reprojection.
pandas>=2.0.0	Attribute tables and numeric summaries.
numpy>=1.24.0	Raster arrays and statistics.
rasterio>=1.3.0	Raster read/write, transforms, sampling.
scipy>=1.10.0	Convolution for raster smoothing.
matplotlib>=3.7.0	Visualisation support.
folium>=0.15.0	Interactive HTML web map export.
contextily>=1.3.0	Basemap tiles for exports.
reportlab (dev)	Generates this report (PDF).

16. Code Statistics

Metric	Value
data_model.py lines	2236
main.py lines	4461
Total core lines	6697
Test lines (7 scripts)	1954
Classes in data_model.py	5
Module functions in data_model.py	31
Methods in data_model.py	83
Classes in main.py	25
Methods in main.py	237

Metric	Value
Event handlers (on_* in main.py)	93
Dialog classes	23
Menu sections	6 (File, Edit, View, Layer, Raster, Spatial)
Spatial analysis methods (SpatialAnalysisEngine)	6 (suitability, influence zones, corridor vulnerability, least-cost path, cost accu
Raster editing operations	8
Field templates	23 (Blank + 22 presets)
Standardized template format version	1.0 (text / number / integer / boolean)

Class breakdown in data_model.py (methods each):

Class	Methods
Layer	10
Feature	4
MapLayer	29
TerrainLayer	26
SpatialAnalysisEngine	14

Class breakdown in main.py (methods each):

Class	Methods
DetailDialog	2
PropertyDialog	2
SingleFieldDialog	2
NewLayerDialog	6
NewRasterDialog	2
RasterEditDialog	12
RasterPreviewDialog	7
RasterStyleDialog	6
BandAddDialog	2
BandManagerDialog	10
LayerStyleDialog	6
MapCanvasPanel	14
AddFeatureDialog	10
GeometryEditDialog	2

Class	Methods
LayerChoiceDialog	2
SplitCoordinatesDialog	2
SuitabilityAnalysisDialog	3
OrgReachDialog	2
NetworkCostDialog	5
TemplateDialog	5
TemplateNameDialog	2
TemplateLibraryDialog	5
PositionDialog	2
FieldListDialog	28
MainFrame	98

17. Known Limitations & Suggested Next Steps

Known limitations:

- Raster bands embedded in project files are compressed (zlib + base64); very large rasters are decimated into a reduced working copy at load time, so edits then operate on the reduced grid (the original dimensions are kept for reference).
- Raster data is held in memory as float64 (GeoTIFF integer datatypes are read and converted); editing operations always target the active band and leave other bands untouched.
- The map canvas supports per-layer symbology and attribute-driven graduated/categorised styling, but it does not yet support graduated marker sizes or an interactive geographic basemap.
- Terrain layers keep a file reference for reload; if the file is missing only the embedded data is used.
- The legend re-sorts only through explicit Move Up / Move Down actions; there is no drag-and-drop reordering in the legend itself.
- Spatial analysis distances use great-circle (haversine) approximations and Voronoi cells are built in a lon/lat plane clipped to the data bbox, so near the poles or over very large extents results are approximate rather than projected.
- Resource suitability is proximity-based unless a cost surface and weight are supplied (corridor vulnerability still uses great-circle approach distances); cost-weighted network reach is normalized by the largest observed reach cost when no explicit threshold is given, so it is relative within a run rather than an absolute metric. The buffer/Voronoi/density zones are generated in EPSG:4326 without a chosen target projection, and least-cost pathfinding is capped at 4 million cells per raster for responsiveness.
- Template application adds fields but does not rename or migrate existing fields; importing a template onto a layer whose fields already exist is idempotent rather than a merge/replace.

Suggested next steps:

- Support graduated marker sizes and more chart-like legend swatches for attribute-styled layers.

- bullet Allow dragging layers directly in the legend to reorder the draw order.
- bullet Add an interactive geographic basemap to the map canvas.
- bullet Run the resource suitability overlay (and the other spatial analyses) in a projected CRS chosen for the study area, so distances and reach metrics are planar rather than great-circle approximations.
- bullet Add a template editor that lets users build and reorder fields, defaults and layer properties directly in a dialog before saving to the library or a JSON file.
- bullet Let templates be imported with an option to rename/merge into an existing layer's schema rather than only adding new fields.

Appendix: PROJ/GDAL Environment Fix

At the top of data_model.py, before geopandas and rasterio are used, the PROJ_LIB and GDAL_DATA environment variables are pointed at the data folders shipped inside the rasterio package. This prevents the common Windows 'PROJ_LIB not found' / 'Cannot find PROJ database' errors when GDAL data is not installed system-wide. If rasterio is unavailable the script falls back to pyproj's data directory, then silently continues.